

UNIVERSIDADE DE FRANCA

CIÊNCIA DA COMPUTAÇÃO

ALUNO GABRIEL MARQUES PIZA – RGM: 34618783

ALUNO THIAGO BELAGAMBA BUENO – RGM: 34641629

**TraceGuard: Plataforma Unificada de Observabilidade, APM
e Monitoramento de Uptime Self-Hosted**

Pré-projeto de Trabalho de Conclusão de Curso (TCC) apresentado como requisito para a aprovação na etapa preliminar de desenvolvimento de software para o semestre 2026.2.

Orientador: Prof. Me. Euller Da Silva Ferreira

Disciplinas Envolvidas: Engenharia de Software, Sistemas Distribuídos, Banco de Dados, Redes de Computadores e Interação Humano-Computador (IHC).

Franca – SP

2026

Table of Content

1. INTRODUÇÃO	3
2. JUSTIFICATIVA DO IMPACTO SOCIAL DO TRABALHO	4
3. FERRAMENTAS A SEREM UTILIZADAS	5
4. BIBLIOGRAFIA A SER UTILIZADA	7

1. INTRODUÇÃO

As infraestruturas computacionais modernas produzem diariamente um volume massivo de registros brutos e desestruturados. Nesse cenário, incidentes críticos acabam se misturando a ruídos operacionais rotineiros, a exemplo de bloqueios de CORS ou exceções previstas em regras de negócio. Esse excesso de informações irrelevantes culmina na "Fadiga de Alertas" (Alert Fatigue), fenômeno que congestiona os canais de comunicação das equipes técnicas e provoca atrasos severos na identificação e resolução das causas originais de falhas sistêmicas.

Somado a esses desafios de acompanhamento e triagem, destaca-se a vulnerabilidade dos dados face à extração automatizada em massa não autorizada (Web Scraping), prática que favorece fraudes e quebras de privacidade. Para contornar essa ineficiência crônica na gestão de sistemas online, propõe-se o desenvolvimento do TraceGuard: uma Plataforma Unificada de Observabilidade, APM (Application Performance Monitoring) e Monitoramento de Disponibilidade autohospedada (Self-Hosted). A solução tem como objetivo reduzir as interrupções operacionais (downtime) e integrar recursos de proteção de borda em tempo de execução (RASP) a uma inspeção profunda do código-fonte.

O projeto é direcionado principalmente a equipes de desenvolvimento de software, engenheiros de confiabilidade de sites (SREs) e gestores de TI de pequenas e médias empresas (PMEs). Trata-se de um segmento corporativo que, frequentemente, não dispõe de capital para arcar com as licenças onerosas das ferramentas proprietárias de mercado. Por meio do seu formato de implantação independente, a plataforma democratiza recursos avançados de observabilidade a um baixo custo. Consequentemente, além de garantir a resiliência de arquiteturas tecnológicas críticas, o trabalho favorece indiretamente a sociedade e o consumidor final, promovendo o uso de serviços digitais mais contínuos, seguros e robustos no dia a dia.

2. JUSTIFICATIVA DO IMPACTO SOCIAL DO TRABALHO

A relevância deste projeto reside na democratização do acesso a ferramentas de observabilidade avançada, recursos historicamente restritos a corporações com elevados orçamentos de TI. Pequenas e médias empresas (PMEs) frequentemente carecem de capacidade financeira para custear as licenças proibitivas de soluções proprietárias hegemônicas no mercado. Consequentemente, suas equipes de engenharia e de confiabilidade de sites (SREs) lidam com o esgotamento profissional causado pelo processamento ineficiente de milhares de logs desestruturados. Ao adotar um modelo de distribuição autohospedado (self-hosted), o TraceGuard mitiga a saturação dos canais de comunicação e viabiliza uma gestão de incidentes economicamente sustentável, permitindo que as equipes seccionem ruídos operacionais de travamentos críticos com agilidade.

Além da otimização das rotinas de manutenção, o trabalho justifica-se pela necessidade premente de assegurar a integridade de aplicações contra ameaças contemporâneas. A ausência de proteção unificada facilita práticas como a extração automatizada não autorizada de informações em massa (Web Scraping), que atua como vetor primário para fraudes e violações de privacidade. Ao integrar mecanismos de proteção de borda em tempo de execução (RASP) ao monitoramento profundo de código (APM), a plataforma estabelece uma blindagem proativa. Essa abordagem reduz drasticamente o tempo de inatividade (downtime) e garante a estabilidade de infraestruturas tecnológicas críticas.

Por fim, o impacto da solução transcende o ambiente corporativo e alcança, em última instância, a sociedade de forma ampla. Ao capacitar desenvolvedores e gestores com tecnologias precisas de diagnóstico, o software contribui diretamente para a disponibilidade contínua, a segurança e a confiabilidade dos serviços digitais utilizados rotineiramente pela população. Assim, o projeto não apenas soluciona

dores técnicas e financeiras agudas do setor de engenharia de software, mas também atua como um pilar de resiliência para o ecossistema digital moderno.

3. FERRAMENTAS A SEREM UTILIZADAS

Para o desenvolvimento do TraceGuard, selecionou-se um ecossistema tecnológico focado em alto desempenho, escalabilidade e processamento assíncrono. A arquitetura preliminar propõe uma divisão clara entre as camadas de telemetria, ingestão, mensageria, armazenamento e visualização. Cabe ressaltar que as tecnologias aqui elencadas representam a configuração estrutural inicial da plataforma, mantendo-se a flexibilidade arquitetônica para eventuais adaptações, substituições ou otimizações conforme a evolução dos requisitos e as demandas específicas levantadas durante o desenvolvimento.

As principais tecnologias e ferramentas propostas incluem:

- **Agente de Telemetria (SDK/Biblioteca):** A ser desenvolvido em linguagem TypeScript sob o ecossistema Node.js. A escolha justifica-se pela possibilidade de utilizar as APIs nativas de baixo *overhead* `async_hooks` (essenciais para o gerenciamento de contextos isolados e propagação de *Trace IDs*) e `perf_hooks` (para cronometragem de alta precisão).
- **Servidor Backend de Ingestão:** Construído em Node.js com TypeScript. A tecnologia foi selecionada por sua eficiência nativa em operações de entrada e saída (I/O) não bloqueantes, permitindo a implementação de uma arquitetura orientada a eventos para a recepção assíncrona de lotes (*batches*) de *logs* e métricas.
- **Camada de Mensageria e Filas:** Utilização do *broker* RabbitMQ. Sua inclusão é estratégica para garantir o amortecimento de picos de carga de telemetria, assegurando a resiliência de todo o ecossistema e prevenindo o travamento da aplicação que está sendo monitorada.
- **Banco de Dados:** O armazenamento persistente será estruturado inicialmente no sistema gerenciador de banco de dados relacional PostgreSQL, utilizando

otimizações em índices e *views* materializadas para análise temporal. Ressalta-se que o PostgreSQL passará por rigorosos testes de estresse para validação de sua capacidade de ingestão massiva de séries temporais (*time-series*). A equipe mantém-se aberta à possibilidade de pivotar a arquitetura de persistência para um banco de dados especializado nesse tipo de modelagem, caso a demanda exija.

- **Interface Visual (Dashboard):** Construída utilizando o *framework* Next.js / React, combinado ao Tailwind CSS. Estas ferramentas foram escolhidas por permitirem a criação de um *design* responsivo e ágil, focado na visualização de dados em tempo real mediante a integração com *WebSockets* (para sincronização de alertas e testes de *uptime*).
- **Infraestrutura e Deploy:** Todo o ecossistema central será empacotado e orquestrado através de contêineres Docker e Docker Compose. Essa abordagem foi escolhida por simplificar e padronizar o processo de distribuição no modelo autohospedado (*self-hosted*).
- **Ambientes de Desenvolvimento (IDEs):** Serão utilizados o Cursor e o Visual Studio Code, ferramentas selecionadas pelo amplo suporte ao ecossistema JavaScript/TypeScript e pelas extensões que otimizam a produtividade da equipe.

4. BIBLIOGRAFIA A SER UTILIZADA

BEYER, Betsy; JONES, Chris; PETERMAN, Jennifer; MURPHY, Niall Richard. Site Reliability Engineering: How Google Runs Production Systems. 1. ed. Sebastopol: O'Reilly Media, 2016.

KLEPPMANN, Martin. Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems. 1. ed. Sebastopol: O'Reilly Media, 2017.

NODE.JS. Asynchronous Context Tracking (AsyncLocalStorage) Documentation. [S. l.]: Node.js Foundation, 2024. Disponível em: https://nodejs.org/api/async_context.html.

OPENTELEMETRY. OpenTelemetry Specifications and Core Concepts. [S. l.]: OpenTelemetry, 2025. Disponível em: <https://opentelemetry.io/docs/>.

OWASP Foundation. OWASP API Security Top 10. [S. l.]: OWASP Project, 2023. Disponível em: <https://owasp.org/www-project-api-security/>.